A Appendix

A.1 Omitted Proofs

Proof (Proof of Theorem 1) To compute the neighbor set $N(e)$ of a hyperedge e, we iterate over all vertices $u \in e$ and enumerate the incident hyperedges in $E(u)$ to identify candidate neighbors e'. Since $|e| \leq \delta$ and $|E(u)| \leq d$ for any $u \in \mathcal{V}$, the above procedure takes $O(\delta d)$ time for a single hyperedge e. The worst case of this online search needs to traverse the whole hypergraph, applying this procedure to all m hyperedges yields the total time complexity $O(m\delta d)$.

Proof (Proof of Lemma 1) This can be easily derived according to the concept of Definition 7.

Proof (Proof of Lemma 2) ($\Rightarrow$) If e_w transitively covers $\{u, e, s\}$, then $e_w \overset{s}{\rightsquigarrow} e$ and $\mathcal{O}(e_w) < \mathcal{O}(e)$. Since $e_w \overset{s}{\rightsquigarrow} e$ and $e \overset{s}{\rightsquigarrow} e_u$, we have $e_w \overset{s}{\rightsquigarrow} e_u$. Therefore, e_w transitively covers $e \overset{s}{\rightsquigarrow} e_u$.

($\Leftarrow$) If e_w transitively covers $e \overset{s}{\rightsquigarrow} e_u$, then $e \overset{s}{\rightsquigarrow} e_w$, $e_w \overset{s}{\rightsquigarrow} e_u$ and $\mathcal{O}(e_w) < \mathcal{O}(e)$. Since $e_w \overset{s}{\rightsquigarrow} e_u$ and $e_u \in E(u)$, we have $u \overset{s}{\rightsquigarrow} e_w$. Therefore, e_w transitively covers $\{u, e, s\}$.

Proof (Proof of Lemma 3) *i*) If e_u is of the highest importance in $W(e, e_u)$, that is, $\mathcal{O}(e_u) < \mathcal{O}(e)$, clearly, e_u transitively covers $\{u, e, s\}$.

ii) If e_w is of the highest importance in $W(e, e_u)$ and $e_w \neq e$, $e_w \neq e_u$, then $\mathcal{O}(e_w) < \mathcal{O}(e)$ and $\mathcal{O}(e_w) < \mathcal{O}(e_u)$. Since $WOD(W(e, e_u)) \geq s$, we have $e_u \overset{s}{\rightsquigarrow} e_w$ and $e_w \overset{s}{\rightsquigarrow} e$. Therefore, $u \overset{s}{\rightsquigarrow} e_w$, and e_w transitively covers $\{u, e, s\}$.

Proof (Proof of Theorem 2) Suppose there exists a pair of vertices u and v such that $MR(u, v) = s$ cannot be correctly answered using $\mathcal{L}$. By definition, there exists a walk $W(e_u, e_v)$ with $u \in e_u$, $v \in e_v$, and $WOD(W(e_u, e_v)) = s$. Let e be the most important hyperedge among all such walks. Then there exist s_u, s_v such that $e \overset{s_u}{\rightsquigarrow} e_u$ and $e \overset{s_v}{\rightsquigarrow} e_v$, where $\min(s_u, s_v) = s$. Since the query cannot be correctly answered, $(e, s_u) \notin \mathcal{L}(u)$ or $(e, s_v) \notin \mathcal{L}(v)$. Without loss of generality, assume $(e, s_u) \notin \mathcal{L}(u)$. By Algorithm 2, this implies that (e, s_u) is transitively covered by a hyperedge e_w with higher importance than e, i.e., $e_w \overset{s_u}{\rightsquigarrow} e$ and $e_w \overset{s_u}{\rightsquigarrow} e_u$. Combining with $e \overset{s_v}{\rightsquigarrow} e_v$, we obtain a walk from e_w to e_v with overlapping degree at least $\min(s_u, s_v) = s$. This contradicts the assumption that e is the most important hyperedge among all such walks.

Proof (Proof of Lemma 4) ($\Rightarrow$) If there exists a hyperedge e_w with $\mathcal{O}(e_w) < \mathcal{O}(e)$ that transitively covers $e \overset{s}{\rightsquigarrow} e_u$, then $e_w \overset{s}{\rightsquigarrow} e$, that is, there exists a walk $W(e_w, e)$ with $WOD(W(e_w, e)) = s$. Therefore, following the definition of maximum cover degree, $MCD(e) \geq s$.

($\Leftarrow$) Suppose $MCD(e) = k \geq s$, based on the definition of maximum cover degree, there exists a walk $W(e_w, e)$ with $\mathcal{O}(e_w) < \mathcal{O}(e)$ and $WOD(W(e_w, e)) = s$. Given $e \overset{s}{\rightsquigarrow} e_u$, i.e., there exists a walk $W(e, e_u)$ with $WOD(W(e, e_u)) = s$, then we can form a walk $W(e_w, e_u) = W(e_w, e) \oplus W(e, e_u)$ with $WOD(W(e_w, e_u)) = \min(k, s) = s$, that is, $e_w \overset{s}{\rightsquigarrow} e_u$. Together with $e_w \overset{s}{\rightsquigarrow} e$, we can conclude $e \overset{s}{\rightsquigarrow} e_u$ is transitively covered by e_w.

Proof (Proof of Lemma 5) Based on Lemmas 2, 3 and the construction workflow mentioned in Algorithm 2, we early terminated the walk W that reaches hyperedge e when *i*) it only brings non-dominant tuples, and *ii*) it has been transitively covered by another hyperedge. Both cases indicate that we have explored another walk W' that reaches e with $WOD(W') \geq WOD(W)$. Suppose there exists a walk $W(e', e)$ that has been pruned in Algorithm 2, which affects the $MCD(e)$. According to Definition 8, this $W(e', e)$ has the highest walk overlapping degree among all walks in $\mathcal{W}(e_w, e)$ for all hyperedges e_w that satisfy $\mathcal{O}(e_w) < \mathcal{O}(e)$, which contradicts the pruning conditions in Algorithm 2, so the lemma holds.

Proof (Proof of Lemma 6) Suppose $WOD(W(e', e_u)) = s_1$ and $OD(e_u, e_v) = s_2 \leq k$, then $s = \min(s_1, s_2)$. Since $e \overset{k}{\rightsquigarrow} e_u$, there exists a walk $W(e, e_u)$ with $WOD(W(e, e_u)) = k$, and we can form two walks $W(e', e) = W(e', e_u) \oplus W(e_u, e)$ and $W(e, e_v) = W(e, e_u) \oplus \{e_v\}$. When $s_1 \geq s_2$, then we have $s = s_2$, and $WOD(W(e, e')) = \min(k, s_2) = s_2 = s$. Therefore, the lower bound of $MCD(e')$ is no smaller than s_2, so such $W(e', e_u) \oplus e_v$ will be early terminated at e_u. Another case is $s_1 < s_2$, in this case, the lower bound of $MCD(e')$ is s_1 derived from the walk $W(e, e_u) \oplus W(e_u, e')$, and such $W(e, e_u)$ with $WOD(W(e', e_u)) \leq MCD(e')$ will be early terminated. In both cases $e \overset{s}{\rightsquigarrow} e'$ and $e \overset{s}{\rightsquigarrow} e_v$, therefore $e' \overset{s}{\rightsquigarrow} e_v$ is transitively covered by e, so it holds.

Proof (Proof of Lemma 7) This can be easily derived according to the completeness property of the HL-index, so we omit the details.

Proof (Proof of Theorem 3) The time complexity of Algorithm 3 can be categorized into three parts, and we analyze them separately. *i*) Graph traversal: We start by analyzing the total number of elements pushed onto the priority queues. During the index construction from a hyperedge e, a label $(e, s) \in \mathcal{L}(u)$ is inserted (line 12) when u is first visited from e, that is, a hyperedge e_u

that contains u is popped from the priority queue and being explored. Since each e_u is explored at most once from e, in the worst case, we explore all hyperedges in $E(u)$ and are only able to insert one label into $\mathcal{L}$. Thus, the total number of elements pushed onto and popped from the priority queues is bounded by $O(ld)$, and each push/pop operation takes $O(\log(ld))$ time given that the size of priority queues is at most $O(ld)$. Therefore, lines 7 and 21 take $O(ld \cdot \log(ld))$ time in total. Each time a hyperedge e_u is popped from the priority queues, line 10 iterates through all vertices in e_u and line 9 scans the neighbor-index of e_u, which takes $O(ld \cdot (\delta + \alpha_e))$ time in total. *ii*) Index update: Line 12 inserts a new label (e, s) into $\mathcal{L}(u)$ by appending it to the end of $\mathcal{L}(u)$, which takes constant time for each insertion and the overall index updating time for $\mathcal{L}$ is $O(l)$. *iii*) neighbor-index maintenance: The neighbor information for each hyperedge is computed exactly once (line 15), and the computation time is bounded by $O(m\delta d)$. The overall time complexity for deleting elements in neighbor-index is bounded by $O(\alpha \log \alpha_e)$. Therefore, the overall time complexity of Algorithm 3 is bounded by $O(ld \cdot (\log(ld) + \delta + \alpha_e) + m\delta d + \alpha \log \alpha_e)$.

Proof (Proof of Theorem 4) According to Definition 5 and Lemma 3, we only add a label (e, s) into $\mathcal{L}(u)$ when e forms a walk $W(e, e_u)$ such that $u \in e_u$ and e has the highest importance among all hyperedges in $W(e, e_u)$. Note that, e_u can be e. Therefore, the number of labels in $\mathcal{L}(u)$ will be no greater than $|\mathcal{E}_{\leq u}|$ and the overall space complexity for our HL-index in both Algorithm 2 and 3 is bounded by $O(\sum_{u \in \mathcal{V}}(|\mathcal{E}_{\leq u}|))$. For a hyperedge e and the corresponding neighbor-index $\mathcal{M}(e)$, only those e_v with $OD(e, e_v) > MCD(e)$ will be maintained in $\mathcal{M}(e)$, and the number of neighbors of e maintained in $\mathcal{M}(e)$ will be a value $\alpha_e \leq \eta_{max}$. So the space complexity of the neighbor-index $\mathcal{M}$ is bounded by $O(m\alpha_e)$. For the neighbor-index, consider a hyperedge e. Since $OD(e, e_v) = |e \cap e_v|$, we have $\sum_{e_v \in N(e)} OD(e, e_v) = \sum_{v \in e}\big(|E(v)| - 1\big)$. During initialization, $\mathcal{M}(e)$ stores only neighbors satisfying $OD(e, e_v) > MCD_{\text{init}}(e)$, and subsequent updates only remove entries. Since overlapping degrees are integers, every retained neighbor satisfies $OD(e, e_v) - 1 \geq MCD_{\text{init}}(e)$. Every other neighbor satisfies $OD(e, e_v) - 1 \geq 0$. Thus,

$$\begin{aligned} MCD_{\text{init}}(e)|\mathcal{M}(e)| &\leq \sum_{e_v \in N(e)} \big(OD(e, e_v) - 1\big) \\ &= \sum_{v \in e}\big(|E(v)| - 1\big) - |N(e)|. \end{aligned}$$

The initial MCD value is positive. Dividing by $MCD_{\text{init}}(e)$ and taking the maximum over all hyperedges and construction stages yields

$$\alpha_e \leq \max_{e \in \mathcal{E}} \frac{\sum_{v \in e}\big(|E(v)| - 1\big) - |N(e)|}{MCD_{\text{init}}(e)}.$$

Each of the m neighbor lists stores at most α_e entries, so the neighbor-index requires $O(m\alpha_e)$ space.

Proof (Proof of Theorem 5) Since $\sum_{e \in \mathcal{E}} |\mathcal{D}(e)| = l$, the outer loops in lines 4 and 7 are executed l times in total. For each $(v, s_v) \in \mathcal{D}(e)$, the loop in line 5 scans at most l_v labels in $\mathcal{L}(v)$. Hence, the initialization of $\mathcal{I}$ in lines 4–6 takes $O(l \cdot l_v)$ time. For each $(u, s_u) \in \mathcal{D}(e)$ processed in line 7, the loop in line 9 examines at most l_v labels in $\mathcal{L}(u)$, and for each candidate e', the loop in line 10 scans at most $|\mathcal{I}(e')| \leq \beta$ entries. By Observation 1, $\mathcal{I}(e') \subseteq \mathcal{D}(e')$, thus $\beta \leq \theta$. Since the membership test and insertion operations in lines 11-12 each take $O(\log \theta)$ time, lines 9-13 require $O(l \cdot l_v \beta \log \theta)$. For the remaining part, line 14 takes $O(\log \theta)$ time due to the search on $\mathcal{D}(e)$. Line 15 takes $O(1)$ time, since (e, s_u) can be directly appended to $\mathcal{L}^*(u)$ under the descending hyperedge order. Line 16 performs set updates on at most θ vertices, and thus costs $O(\theta \log \theta)$. Therefore, lines 14–18 contribute $O\big(l(\log \theta + \theta \log \theta)\big)$. Combining the above bounds, the overall time complexity of Algorithm 4 is $O\big(l \cdot l_v \beta \log \theta + l(\log \theta + \theta \log \theta)\big) = O\big(l \cdot (l_v \beta + \theta) \log \theta\big)$.

Proof (Proof of Theorem 6) Evidently, Algorithm 4 will remove a label (e, s) from $\mathcal{L}(u)$ if and only if $MR(u, v)$ query for any $v \in \mathcal{V}$ can be correctly answered with other labels in the current $\mathcal{L}$.

First, we prove the completeness of $\mathcal{L}^*$ as follows. Suppose $MR(u, v) = k > 0$ cannot be correctly determined through $\mathcal{L}^*$, that is, there does not exist a hyperedge that supports $MR(u, v)$ in $\mathcal{L}^*$. According to Theorem 2, the input $\mathcal{L}$ is complete, therefore, there exists at least one hyperedge e that supports $MR(u, v)$ in the input $\mathcal{L}$ and we denote the set of such hyperedges as $\mathcal{E}_{u,v}$. $MR(u, v)$ cannot be derived from $\mathcal{L}^*$ only if the support for $MR(u, v)$ from each hyperedge $e \in \mathcal{E}_{u,v}$ is disrupted in Algorithm 4 when its essential tuples are being verified and $(e, *)$ is deleted either from $\mathcal{L}(u)$ or from $\mathcal{L}(v)$. Let the hyperedge e' be the last hyperedge in $\mathcal{E}_{u,v}$ to have its essential tuples verified. Then, the support from e' is disrupted if and only if $MR(u, v)$ is still supported by other hyperedges from $\mathcal{E}_{u,v}$ in the current $\mathcal{L}$, which contradicts our assumption. Therefore, $\mathcal{L}^*$ is complete.

Next, we prove $\mathcal{L}^*$ satisfies the minimality property. Suppose there exists a label $(e, s) \in \mathcal{L}^*(u)$ such that, for all $v \in \mathcal{V}$, $\mathcal{L}^*$ can still correctly answer $MR(u, v)$ after we remove (e, s) from $\mathcal{L}^*(u)$. Then, Algorithm 4 would have already removed (e, s) from $\mathcal{L}$, which contradicts our assumption. Therefore, $\mathcal{L}^*$ is minimal.

Proof (Proof of Theorem 7) Both $\mathcal{L}(u)$ and $\mathcal{L}(v)$ are stored in descending order of hyperedge importance, since Algorithm 3 constructs hyperedges in that order and appends each new entry to the end of the corresponding list. Algorithm 5 therefore scans the two lists with a merge procedure, advancing the pointer of the list whose current entry is less important, and comparing the overlapping degrees whenever the two pointers refer to the same hyperedge. Each entry is visited at most once, and no entry is revisited, so the time complexity of Algorithm 5 is $O(|\mathcal{L}(u)| + |\mathcal{L}(v)|)$.

Proof (Proof of Lemma 8) It can be easily derived from the HL-index completeness property.

Proof (Proof of Theorem 8) Suppose there exists a pair of hyperedges e_i and e_j such that $MR(e_i, e_j) = s$ cannot be correctly answered using $\mathcal{L}_e$. By definition, there exists a walk $W(e_i, e_j)$ with $WOD(W(e_i, e_j)) = s$. Let e_w be the most important hyperedge among all such walks. Then there exist s_i, s_j such that $e_w \overset{s_i}{\rightsquigarrow} e_i$ and $e_w \overset{s_j}{\rightsquigarrow} e_j$, where $\min(s_i, s_j) = s$. Since the query cannot be correctly answered, $(e_w, s_i) \notin \mathcal{L}_e(e_i)$ or $(e_w, s_j) \notin \mathcal{L}_e(e_j)$. Without loss of generality, assume $(e_w, s_i) \notin \mathcal{L}_e(e_i)$. By Algorithm 3, this implies that (e_w, s_i) is transitively covered by a hyperedge e_x with higher importance than e_w, i.e., $e_x \overset{s_i}{\rightsquigarrow} e_w$ and $e_x \overset{s_i}{\rightsquigarrow} e_i$. Combining with $e_w \overset{s_j}{\rightsquigarrow} e_j$, we obtain a walk from e_x to e_j with overlapping degree at least $\min(s_i, s_j) = s$. This contradicts the assumption that e_w is the most important hyperedge among all such walks.

Proof (Proof of Lemma 9) Note that a strengthened walk containing $W(e_u, e_v)$ has a walk overlapping degree of at most s, and only those with walk overlapping degree s carry the s-reachability between e_u and e_v. Following the notation W^s, the strengthened walks considered in Definition 10 are therefore those with walk overlapping degree s. Suppose there exists a non-pivot triplet (e_u, e_v, s), derived from a strengthened walk $W(e_u, e_v)$, whose max-reachability information cannot be derived from pivot triplets. Let e_w be the most important hyperedge that is s-reachable from e_u after the insertion. Since $e_u \overset{s}{\rightsquigarrow} e_v$, e_w is also the most important hyperedge that is s-reachable from e_v. Since (e_u, e_v, s) is not a pivot triplet, some strengthened walk with walk overlapping degree s containing $W(e_u, e_v)$ has a hyperedge more important than both e_u and e_v. This hyperedge is s-reachable from e_u, so we have $\mathcal{O}(e_w) < \mathcal{O}(e_u)$ and $\mathcal{O}(e_w) < \mathcal{O}(e_v)$. Consider $e_u \overset{s}{\rightsquigarrow} e_w$, and the case of e_v is symmetric. If it already holds before the insertion, it is maintained in $\mathcal{L}_e$ by Theorem 8. Otherwise, since the max-reachability increases by at most one, it is established by a strengthened walk $W(e_u, e_w)$ with walk overlapping degree s. Every hyperedge on a strengthened walk with walk overlapping degree s containing $W(e_u, e_w)$ is s-reachable from e_u, and is thus no more important than e_w. Hence (e_u, e_w, s) is a pivot triplet. By Definition 6, $\{e_u, e_v, s\}$ is transitively covered by e_w, and is therefore derivable from pivot triplets together with the reachability already maintained before the insertion, which contradicts our assumption.

Proof (Proof of Lemma 10) Consider any non-redundant $W^s(e_w, e)$ that induces $(e_w, s) \in \mathcal{L}_e(e)$ after $Insert(u, e)$. Without loss of generality, e appears only at the end of $W^s(e_w, e)$, since otherwise its prefix ending at the first occurrence of e already establishes $e_w \overset{s}{\rightsquigarrow} e$. Since the insertion only modifies the membership of u in e, the strengthened part of $W^s(e_w, e)$ must pass through some hyperedge e' with $u \in e'$, and hence $e' \in N(e)$. Therefore, $W^s(e_w, e)$ can be decomposed into two sub-walks: $W(e_w, e')$ and $W(e', e)$. Since $W(e_w, e')$ does not contain e, its WOD is not affected by the insertion, and we denote it by s'. Since $W^s(e_w, e)$ is strengthened by the insertion, its bottleneck must lie on the last hop, as otherwise its WOD would be unchanged. Thus s equals the overlapping degree between e and e' after the insertion, and $s' \geq s$. The equality may hold, for example when $e' = e_w$ and $|e_w| = OD(e, e_w)$ after the insertion. Moreover, e_w is explicitly stored in $\mathcal{L}_e(e')$ with a value no less than s', since if $e' \overset{s'}{\rightsquigarrow} e_w$ were covered by a more important hyperedge e'', then e'' would also cover $W^s(e_w, e)$, contradicting its non-redundancy. So the lemma holds.

Proof (Proof of Lemma 11) Consider any non-redundant $W^s(e_w, e)$ that induces $(e_w, s) \in \mathcal{L}_e(e)$ after $Insert(u, e)$. Without loss of generality, e appears only at the end of $W^s(e_w, e)$, since otherwise its prefix ending at the first occurrence of e already establishes $e_w \overset{s}{\rightsquigarrow} e$. Since the insertion only modifies the membership of u in e, the strengthened part of $W^s(e_w, e)$ must pass through some hyperedge e' with $u \in e'$, and hence $e' \in N(e)$. Therefore, $W^s(e_w, e)$ can be decomposed into two sub-walks: $W(e_w, e')$ and $W(e', e)$. Since $W(e_w, e')$ does not contain e, its WOD is not affected by the insertion, and we denote it by s'. Since $W^s(e_w, e)$ is strengthened by the insertion, its bottleneck must lie on the last hop, as otherwise its WOD

would be unchanged. Thus s equals the overlapping degree between e and e' after the insertion, and $s' \geq s$. The equality may hold, for example when $e' = e_w$ and $|e_w| = OD(e, e_w)$ after the insertion. Moreover, e_w is explicitly stored in $\mathcal{L}_e(e')$ with a value no less than s', since if $e' \overset{s'}{\rightsquigarrow} e_w$ were covered by a more important hyperedge e'', then e'' would also cover $W^s(e_w, e)$, contradicting its non-redundancy. So the lemma holds.

Proof (Proof of Lemma 12) Let $W(e', e_w)$ be the strengthened walk deriving (e_w, e', s), whose walk overlapping degree increases from $s-1$ to s. Since $Insert(u, e)$ only increases $|e|$ and $OD(e, e_{nbr})$ for $e_{nbr} \in nbr$ by one, the consecutive hyperedges on $W(e', e_w)$ with overlapping degree $s-1$ before the insertion must be e and some $e_{nbr} \in nbr$, and thus $OD(e, e_{nbr}) = s$ after the insertion. Moreover, $MR(e', e_w) = s-1$ before the insertion. By Theorem 8, there exists e_x with $(e_x, s_1) \in \mathcal{L}_e(e')$, $(e_x, s_2) \in \mathcal{L}_e(e_w)$ and $\min(s_1, s_2) = s-1$. If $e_x = e_w$, then e' is explicitly stored in $\mathcal{D}_e(e_w)$. Otherwise $\mathcal{O}(e_x) < \mathcal{O}(e_w)$. If $s_2 \geq s$, extending $W(e', e_w)$ to e_x yields a strengthened walk containing e_x with $\mathcal{O}(e_x) < \mathcal{O}(e_w) < \mathcal{O}(e')$, which contradicts Definition 10. Hence $s_2 = s-1$ and $s_1 \geq s-1$, so e' is stored in $\mathcal{D}_e(e_x)$ with value no less than $s-1$. Since $\mathcal{L}_e(e_w)$ contains no dominated entry, e_x is the unique, and thus the most important, hyperedge with $(e_x, s-1) \in \mathcal{L}_e(e_w)$. This argument does not depend on whether $MR(e, e_w)$ changes, so the lemma holds.

Proof (Proof of Theorem 9) We analyze Algorithm 6 by grouping its operations into three parts. First, lines 2-3 of Algorithm 6 identify the hyperedges in nbr, and PivotsEndingAt updates $\mathcal{L}_e(e)$ using the entries of those neighboring hyperedges e' whose overlapping degree with e increases after the insertion. Since $|E(u)| \leq d$, the number of such neighboring hyperedges is at most d. For each of them, we scan at most l_e entries in $\mathcal{L}_e(e')$, and each max-reachability check or entry update takes at most $O(l_e + \log l_e)$ time. Hence, this part takes $O\big(d\, l_e(l_e + \log l_e)\big)$. Second, PivotsThrough enumerates the pivots that pass through e. The loop in line 9 visits at most $d\, l_e$ pairs of $(e_w, s_w) \in \mathcal{L}_e(e)$ and $e_{nbr} \in nbr$. For each pair, finding e_x in $\mathcal{L}_e(e_w)$ takes $O(l_e)$ time, and $\mathcal{D}_e(e_w) \cup R$ contains at most $2d_e$ candidates. For each candidate e', $MR(e, e')$ after the insertion is obtained by at most $d+1$ max-reachability queries on the ETE-index, each taking $O(l_e)$ time, and the entry update takes $O(\log(l_e d_e))$ time. Therefore, this part takes $O\big(d\, l_e d_e(d\, l_e + \log(l_e d_e))\big)$. Third, PivotsNewlyConnected and UpdateIndexByPivots process the newly connected components and update the HL-index. The number of candidate hyperedges in N_{new} is at most d, and for each selected hyperedge we scan at most d_e entries in the corresponding dual structure. Together with the previous parts, at most $O(d\, l_e d_e)$ pivots are generated. For every generated pivot, updating $\mathcal{L}_e$ and $\mathcal{D}_e$ and propagating the entry to all vertices in a hyperedge costs at most $O(\delta + \log(l_e d_e))$, and discarding the entries that become redundant scans at most l_e entries for each updated hyperedge. Hence, this part is bounded by $O\big(d\, l_e d_e(\delta + l_e + \log(l_e d_e))\big)$. Combining the above three bounds and merging lower-order terms, the total time complexity of Algorithm 6 is bounded by $O\big(d\, l_e d_e(\delta + d\, l_e + \log(l_e d_e))\big)$.

Proof (Proof of Theorem 10) Throughout this proof, a subscript $\mathcal{H}$ or $\mathcal{H}'$ indicates the hypergraph in which max-reachability and walk overlapping degree are evaluated. Given a complete HL-index $\mathcal{L}$ at the beginning. Suppose that after $Insert(u, e)$, $MR_{\mathcal{H}'}(x, y) = s$ cannot be correctly answered using $\mathcal{L}$ for some vertices x, y (violates completeness mentioned in (i)). As Algorithm 6 only inserts entries into $\mathcal{L}$ and line 31 of Algorithm 7 discards only dominated entries, we have $s > MR_{\mathcal{H}}(x, y)$.

Let $W(e_x, e_y)$ be a walk with $x \in e_x$, $y \in e_y$ and $WOD_{\mathcal{H}'} = s$. Since only $|e|$ and $OD(e, \cdot)$ change, any walk avoiding e has $WOD_{\mathcal{H}'} = WOD_{\mathcal{H}} \leq MR_{\mathcal{H}}(x, y) < s$, so $W(e_x, e_y)$ is a strengthened walk through e. Let e_w be the most important hyperedge on $W(e_x, e_y)$. Definition 10 implies that splitting $W(e_x, e_y)$ at e_w yields pivots (e_x, e_w, s_x) and (e_y, e_w, s_y), where $e_x \overset{s_x}{\rightsquigarrow} e_w$, $e_w \overset{s_y}{\rightsquigarrow} e_y$ and $\min(s_x, s_y) = s$. Since the query cannot be correctly answered, we have $(e_w, s_x) \notin \mathcal{L}(x)$ or $(e_w, s_y) \notin \mathcal{L}(y)$.

We then show that Algorithm 6 adds every pivot derived from a strengthened walk to *pivots*, where all line numbers below refer to Algorithm 7. Since e lies on every strengthened walk, each pivot falls into exactly one of four cases, as discussed in Section 6: if e is an endpoint, Lemma 10 guarantees the pivot is found by scanning $\mathcal{L}_e(e_{nbr})$ for $e_{nbr} \in E(u)$ (lines 2-6). If the walk contains e itself only, the pivot is added in line 7. If e is interior, Lemmas 11 and 12 guarantee that the other endpoint is retrieved from $\mathcal{D}_e(e_w)$ or from $\mathcal{D}_e(e_z)$, where e_z is the most important hyperedge with $(e_z, OD(e, e_{nbr}) - 1) \in \mathcal{L}_e(e_w)$ for some $e_{nbr} \in nbr$, regardless of whether $MR(e, e_w)$ changes (lines 9-15); and if its endpoints were previously disconnected, the pivot is generated via max_{nbr} (lines 19-21). Therefore, all pivots, including (e_x, e_w, s_x) and (e_y, e_w, s_y) can always be propagated, and hence insert (e_w, s_x) into $\mathcal{L}(x)$ and (e_w, s_y) into $\mathcal{L}(y)$, since $x \in e_x$ and $y \in e_y$, as shown in lines 28-29. Since $\min(s_x, s_y) = s$, Algorithm 5 returns

s for (x, y), which contradicts our assumption, so (i) holds. (ii) follows from the same enumeration, as each pivot carries its $\mathcal{L}_e$ entry (lines 5, 7, 15, 21 and 25), and (iii) from line 26. So the correctness of Algorithm 6 holds, that is, all indices $\mathcal{L}, \mathcal{L}_e, \mathcal{D}_e$ hold completeness after a single insertion.

Proof (Proof of Observation 2) For every hyperedge $e = \{v\} \in \mathcal{E}$, e cannot transitively cover any other hyperedge since it has the lowest importance among all hyperedges in a walk that contains e. Besides, every walk W that contains e s.t. $W = \{e_1, e, e_2, ..., e_k\}$ can be simplified to $W' = \{e_1, e_2, ..., e_k\}$ while $WOD(W')$ remains unaffected, since $e \cap e_1$ and $e \cap e_2$ are both non-empty and $e = \{v\}$, which gives $v \in e_1$ and $v \in e_2$. The same holds when e is an endpoint of W, in which case $W = \{e, e_2, ..., e_k\}$ is simplified to $W' = \{e_2, ..., e_k\}$.

Proof (Proof of Observation 3) Let u and u' be two vertices with $E(u) = E(u') = \{e\}$. Neither appears in any hyperedge other than e, so neither contributes to any OD value with other hyperedges. Since u and u' are incident to the same e, the walks available from them are identical, so $MR(u, x) = MR(u', x)$ for every vertex x. For u and u' themselves, the walk consisting of e alone gives $MR(u, u') = |e|$, and $|e|$ is recorded when $\mathcal{H}$ is constructed and is not modified by compacting, so this value is still correctly reported after the merge.

A.2 Discussion of 2-hop Index-Based Variants

We discuss and evaluate the VTV-based index, ETE-based index, and VTE-based index from the aspects of correctness, index construction time/space, and query performance. First of all, the VTV-based index cannot guarantee the correctness of the max-reachability query, so we do not consider it as a valid solution. We will mainly compare the difference between the ETE-based index and the VTE-based index. From the space cost aspect, the cost of the VTE-based index can be bounded by Algorithm 2, which is $O(\sum_{u \in \mathcal{V}}(|\mathcal{E}_{\leq u}|))$, where $\mathcal{E}_{\leq u} = \{e | e_u \in E(u) \wedge \mathcal{O}(e) \leq \mathcal{O}(e_u)\}$, as discussed in Theorem 4. The upper bound space cost of ETE-based index can be deduced in a similar way, where for every hyperedge e_u, it stores at most $|\mathcal{E}_{\leq e_u}|$, where $\mathcal{E}_{\leq e_u} = \{e | \mathcal{O}(e) \leq \mathcal{O}(e_u)\}$. The overall space cost for ETE-based index is then $\sum_{e \in \mathcal{E}} |\mathcal{E}_{\leq e_u}|$. Though we have $\mathcal{E}_{\leq e_u} \leq \mathcal{E}_{\leq u}$, in real world hypergraph datasets, $|\mathcal{E}|$ is much larger than $|\mathcal{V}|$. Therefore, the total space costs of the VTE-based index and the ETE-based index are close. For the construction time aspect, when constructing a complete index, the ETE-based index takes a factor of δ since we do not need to iterate through all vertices in e at line 9 of Algorithm 3. From the query efficiency aspects, suppose we are aiming to determine $MR(u, v)$. By adopting the ETE-based index and applying the merge-sort-based algorithm, we should first iterate through all hyperedges in $E(u)$, shuffle and sort all elements inside $\mathcal{L}_u$, which is represented as $\mathcal{L}_u = \bigcup_{e_u \in E(u)} \mathcal{L}_e(e_u)$. Let $l_e^{max} = \max(|\mathcal{L}_e(e_u)|) : e_u \in \mathcal{E}$, building such $\mathcal{L}_u$ takes $O(d \cdot l_e^{max})$, where $d = \max_{v \in \mathcal{V}} |E(v)|$. Construct $\mathcal{L}_v$ for v that affords a similar time cost. Then the ETE-based $MR(u, v)$ query takes $O(|\mathcal{L}_u| + |\mathcal{L}_v|)$, and the overall time cost for ETE-based cost is bounded by $O(d \cdot l_e^{max} + |\mathcal{L}_u| + |\mathcal{L}_v|)$. Generally, $|\mathcal{L}_u| + |\mathcal{L}_v|$ is smaller than l_e^{max}, and for our VTE-based query, it takes $O(|\mathcal{L}(u)| + |\mathcal{L}(v)|)$, where we have $|\mathcal{L}(u)| << |\mathcal{L}_u|$ and $|\mathcal{L}(v)| << |\mathcal{L}_v|$. Therefore, our VTE-based approach outperforms the ETE-based index for answering max-reachability queries in terms of query efficiency.